
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский физико-технический институт
(национальный исследовательский университет)»
Физтех-школа Радиотехники и Компьютерных Технологий
Кафедра микропроцессорных технологий в интеллектуальных системах управления

Направление подготовки / специальность: 03.03.01 Прикладные математика и физика

Направленность (профиль) подготовки: Радиотехника и компьютерные технологии

ИССЛЕДОВАНИЕ И РАЗРАБОТКА ВЕРИФИКАТОРА БАЙТКОДА СТАТИЧЕСКИ ТИПИЗИРОВАННЫХ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ

(бакалаврская работа)

Студент:

Грошев Максим Алексеевич

(подпись студента)

Научный руководитель:

Голенев Александр Дмитриевич,

(подпись научного руководителя)

Консультант (при наличии):

(подпись консультанта)

Москва 2026

Аннотация

Исследование и разработка верификатора байткода статически типизированных языков программирования.

Грошев Максим Алексеевич

В настоящее время большой популярностью пользуются платформонезависимые языки программирования. Их важнейшим преимуществом является переносимость исполняемого кода между различными аппаратными средами. Однако переносимость ставит вопрос о корректности байт-кода. Именно поэтому данная работа своей основной целью ставит исследование и разработку алгоритмов верификации инструкций для виртуальных машин со статической типизацией.

Содержание

1	Введение	4
2	Постановка задачи	6
3	Цели работы:	6
4	Задачи работы:	6
4.1	Ожидаемый результат	6
4.2	Критерии достижения цели	6
5	Обзор существующих решений	8
6	Верификация байт-кода динамически типизированных языков программирования:	8
7	Верификация байт-кода статически типизированных языков программирования:	9
8	Исследование и построение решения задачи	11
9	Устройство верификатора	11
9.1	Режимы верификации:	11
9.2	Схема работы:	12
9.2.1	Проверка графа потока управления	12
9.2.2	Абстрактная интерпретация	13
9.3	plugin opcodes	16
9.3.1	Мотивация появления плагиноспецифических инструкций	17
9.3.2	Основные семейства plugin-инструкций	17
9.3.3	Специальные plugin-инструкции	18
9.4	any opcodes	19
9.4.1	Модель значения <code>any</code>	19
9.4.2	Архитектурная роль значений <code>any</code>	19
9.4.3	Доступ к свойствам и элементам динамических значений	19
9.4.4	Динамические вызовы	21
10	Описание практической части	22
10.1	Подготовка типовой инфраструктуры для рантайм-объединений	22
10.2	Реализация верификации <code>plugin.call.name.*</code>	23
10.3	Реализация верификации <code>any.*</code>	25
10.4	Тестирование результатов	26
10.5	Итоги практической части	26
11	Заключение	28

1 Введение

В настоящее время активно создаются и развиваются языки программирования, ориентированные на разные предметные области и парадигмы разработки. Некоторые языки проектируются под разработку системного программного обеспечения, другие — мобильных приложений или средств автоматизации. При этом значительная часть современных языков относится к платформонезависимым. Их важнейшим преимуществом является переносимость программного кода между аппаратными и программными средами. Данная концепция именуется аббревиатурой *WORA* (*Write Once, Run Anywhere*). К числу таких языков относятся Java, C#, Dart и другие.

Платформонезависимость в подобных системах достигается за счёт использования промежуточного представления программы (байт-кода) и специальной среды исполнения (виртуальной машины). Исходный текст компилируется не в машинный код конкретной архитектуры, а в байт-код — компактный набор инструкций, предназначенный для виртуальной машины. Виртуальная машина, в свою очередь, берет на себя исполнение этого кода, разрешение ссылок на типы и методы, управление памятью и другие функции, которые в нативных системах имеют специфическую реализацию или не имеют вовсе. Такой подход позволяет запускать одну и ту же программу на различных платформах без перекомпиляции, что важно в случае больших приложений и массовых продуктов.

Однако переносимость ставит вопрос о корректности промежуточного кода. В дальнейшем под корректностью байт-кода будем понимать его соответствие архитектуре и семантике набора команд виртуальной машины, для которой этот байт-код был сгенерирован. Байт-код, формально имеющий допустимую структуру, но нарушающий типовые или семантические инварианты платформы, может приводить к ошибкам времени исполнения, нарушению безопасности и некорректному поведению всей программной системы.

Именно поэтому в состав большинства современных платформ входят верификаторы байт-кода — средства статического анализа, проверяющие корректность программы до её фактического выполнения. Назначение верификатора состоит в том, чтобы выявить такие ошибки на как можно более раннем этапе. Это снижает вероятность аварийного завершения программы, уменьшает число трудно воспроизводимых дефектов и повышает общую надёжность платформы. Кроме того, верификатор играет важную роль и в процессе разработки самой виртуальной машины: он позволяет быстрее обнаруживать ошибки компилятора и инфраструктуры байт-кода.

Актуальность таких проверок особенно велика в тех областях, где программное обеспечение работает в условиях повышенных требований к надёжности и безопасности. К таким областям относятся банковская инфраструктура, телекоммуникационные системы, мобильные платформы,

медицинские информационные системы и другие классы программ, где ошибка исполнения может повлечь существенные последствия. В таких условиях наличие точного и достаточно полного верификатора становится не вспомогательной, а принципиально важной частью вычислительной платформы.

Некорректный байт-код может возникать по разным причинам. Наиболее очевидный источник — ошибки компилятора, в том числе на этапах понижения высокоуровневых конструкций, кодогенерации, распределения регистров и формирования служебной информации. Кроме того, байт-код может быть получен не только из доверенного фронтенда: он может быть модифицирован внешними инструментами, испорчен при передаче по сети или сформирован с нарушением ожидаемых инвариантов. Следовательно, полагаться исключительно на корректность компилятора недостаточно, и виртуальная машина должна обладать собственным механизмом независимой проверки.

Особый интерес представляет верификация тех частей байткода, которые отражают нетривиальные свойства языка и рантайма. В случае исследуемой платформы к таким свойствам относятся объединения типов, специальные плагиноспецифические инструкции и операции взаимодействия с динамической средой. Именно такие инструкции часто оказываются наиболее сложными для статического анализа, поскольку соединяют в себе особенности типовой системы языка, соглашения о представлении значений в рантайме и специфику самой архитектуры набора команд.

В данной работе основное внимание уделяется повышению точности верификатора байт-кода платформы за счёт расширения его типовой системы и увеличения числа инструкций, для которых возможна содержательная статическая проверка. Практическая ценность такого подхода состоит не только в расширении множества успешно верифицируемых программ, но и в более точном согласовании верификаторной модели с реальной семантикой исполнения.

2 Постановка задачи

3 Цели работы:

1. Расширить типовую систему верификатора для более точного отображения типовой системы виртуальной машины на типовую систему верификатора.
2. Разработать и внедрить алгоритмы проверок ряда инструкций для расширения множества верифицируемых программ.

4 Задачи работы:

1. Изучить существующие решения.
2. Сравнить типовые системы виртуальной машины и верификатора, выявить неверифицируемые типы.
3. Расширить типовую систему и поддерживать логику обработки данных типов.
4. Проанализировать набор команд платформы и выявить множество неверифицируемых инструкций.
5. Реализовать и внедрить алгоритм верификации новых инструкций.
6. Протестировать каждый из предложенных алгоритмов.
7. Внедрить предложенное решение в программный продукт.

4.1 Ожидаемый результат

Ожидаемым результатом работы является расширение множества верифицируемых программ, которые могут быть проверены до исполнения, а также повышение точности анализа в тех случаях, где прежняя типовая система была недостаточно выразительной. Практически это должно привести к уменьшению числа ложных отказов верификации, более раннему обнаружению ошибок кодогенерации и повышению надёжности механизмов исполнения для плагиноспецифичных и связанных с взаимодействием с динамическими средами инструкций.

4.2 Критерии достижения цели

Цель работы можно считать достигнутой, если верификатор получит поддержку новых инструкций, будет корректно обрабатывать ранее проблемные типовые случаи и подтверждать это на наборе добавленных тестов. Дополнительным критерием является практическая значимость внесённых изменений: способность выявлять ошибки компилятора и улучшать

гарантии платформы по безопасности и корректности исполнения. И если по итогам работы верификация станет включенной по умолчанию в основной открытой ветке разработки платформы, то это будет свидетельствовать о полезности и успешности.

5 Обзор существующих решений

На данный момент большинство виртуальных машин обладают верификаторами, в качестве самых популярных примеров могут послужить JVM - виртуальная машина для исполнения байт-кода таких языков как: Java, Kotlin; .Net CLR - виртуальная машина для исполнения байт-кода C#, F#. Также существуют верификаторы, которые не поставляются вместе с платформой, а являются сторонними библиотеками, примерами могут послужить BCEL Verifier для Java байт-кода и PEVerify, который проверяет на корректность IL-код(C#). Рассмотрим некоторые из существующих решений и виды выполняемых проверок более подробно.

6 Верификация байт-кода динамически типизированных языков программирования:

В языках с динамической типизацией типы определяются во время выполнения программы: переменная может изменять свой тип, а прототипы могут модифицироваться динамически. К тому же во многих языках есть конструкции, принимающие строки кода и выполняющие их уже в рантайме. В связи с этим список возможных проверок ограничен.

1. Проверки формата байткода:

Целью данных проверок является тестирование на валидность инструкций, корректность их формата и правильность индексов, соответствующих методам, полям и типам.

2. Проверки графа управления

Во время выполнения данного типа проверок, программа анализируется на корректность инструкций условного перехода: целевой адрес перехода не выходит за пределы текущего метода, не является серединой try-блока или серединой другой инструкции. Проверяется отсутствие циклов с некорректным состоянием регистров. Контролируется, что все пути выполнения заканчиваются инструкциями, обозначающими *return* или *throw*.

3. Сохранение инварианта стека

Размер стека должен быть неотрицательным числом, при этом его глубина не должна превышать некоторого заданного значения, в точках схождения глубина стека должна быть одинаковой.

4. Проверка инлайн-кэшей:

Inline Cache — это механизм ускорения динамического доступа в JS. Но для безопасного их использования нам надо гарантировать, что при обращении к конкретному слоту он будет существовать и будет

инициализирован, а также что данный слот будет соответствовать типу текущей инструкции.

Важно отметить, что, как правило, движки динамически типизированных языков программирования не имеют такой обособленной компоненты, как верификатор, а перечисленные проверки выполняются непосредственно на этапе выполнения.

7 Верификация байт-кода статически типизированных языков программирования:

В статически типизированных языках типы переменных, аргументов и возвращаемых значений известны на этапе компиляции. Это позволяет компилятору генерировать байткод с встроенными гарантиями типов. Но байткод может быть сгенерирован не только компилятором, поэтому верификатор остается неотъемлемой частью виртуальной машины. Однако теперь возможно выполнение ряда новых проверок вдобавок к тем, что уже были перечислены ранее:

1. Проверки типовой безопасности регистров:

Байткод инспектируется на предмет того, что тип каждого регистра является примитивным или ссылочным. Верифицируется, что любой регистр используется после своей инициализации.

2. Проверки инструкций вызова:

Инструкции вызова проходят проверку на корректность объекта ресивера(в случае нестатического метода). На количество аргументов, на типы данных аргументов и тип возвращаемого значения. Верифицируется возможность доступа к полю или методу класса.

3. Проверка доступа обращения:

Именно данная верификация является одной из самых важных для безопасности. Как известно, существует 3 базовых модификатора доступа: *публичные*, *защищенные* и *приватные*, которые применимы как к элементам класса, так и к классам и целым модулям. Контроль того, что их семантика не нарушается на уровне байткода, необходим.

Рассмотрим пару примеров промышленных решений.

1. ART

ART - Android Runtime В контексте данной работы для нас наибольший интерес представляют верификаторы тех платформ, которые предназначены для исполнения байт-кода на мобильных устройствах. Именно таким является верификатор ART(Android runtime). ART исполняет DEX-байткод, источниками которого могут быть сторонние приложения или злонамеренные источники.

2. LLVM

LLVM - Low level virtual machine LLVM-верификация обладает несколько иной философией, чем ART: она не про безопасность пользователя, а про корректность ПП (здесь и далее ПП - промежуточное представление) компилятора. ПП генерируется доверенным фронтендом, но оптимизации могут его сломать. Можно выделить 4 уровня верификации: уровень модуля, функций, базовых блоков и инструкций. Проверяются как локальные, так глобальные инварианты.

8 Исследование и построение решения задачи

Рассмотрим некоторые теоретические аспекты, которые требуются для выполнения задач данной работы.

9 Устройство верификатора

9.1 Режимы верификации:

Верификация класса может начинаться в различные моменты времени, в зависимости от целей, которые преследуются:

1. Верификация времени установки

Всё приложение, а соответственно его классы, проходят проверку на этапе установки на устройство. Данный подход хорош, когда мы хотим исключить довольно дорогостоящие проверки с этапа исполнения, ускорить старт приложения, получить возможность агрессивной компиляции в машинный код. Однако хоть такой подход и является стандартом индустрии, ультимативным он не является, т.к. имеет ряд недостатков: в данном режиме отсутствует информация о том, какие классы реально будут загружены, какие ветки графа потока управления будут выполнены. Но самой главной проблемой данного подхода является то, что он плохо масштабируется по количеству изменений. Рассмотрим пример: у нас было несколько файлов, они были верифицированы и откомпилированы в ОАТ-файл с машинным кодом, после были внесены какие-то изменения в исходный код, а так как ОАТ-файлы являются цельными артефактами, то они чувствительны к малейшим изменениям, которые могут сломать метайнформацию, и нам приходится перекомпилировать и проводить верификацию, что является довольно энергоёмким процессом.

2. Верификация времени линковки

При данном подходе класс будет верифицирован во время его подготовки к исполнению, которое может быть вызвано, например, созданием его экземпляра. Но т.к. во время верификации будут рекурсивно загружены все зависимые классы, что может привести к деоптимизациям, связанным с девиртуализацией (например, оптимизация иерархии классов (СНА)).

3. Верификация "на лету"

Данный подход является наиболее гибким, верификация будет вызвана только при первом вызове метода. Также данный подход является более инкрементальным, и изменения в конкретном методе приведут только к верификации данного метода или его класса, такой подход больше подходит для JIT-компиляции.

9.2 Схема работы:

Сначала рассмотрим устройство верификатора целевой платформы. Единицей верификации является метод. Для каждого верифицируемого метода создается экземпляр объекта задачи для верифицирования (Job). Процесс создания задачи состоит из разрешения типов входных параметров и возвращаемого типа, построения графа потока управления, а также создания верификаторного контекста, который содержит контекст регистров, являющийся одним из главных используемых инструментов во время символьного исполнения. Разберем каждый из этапов.

9.2.1 Проверка графа потока управления

Верификатор исследуемой виртуальной машины выполняет большинство проверок графа потока управления, которые были перечислены ранее: проверка на то, что адрес перехода - начало инструкции, проверка отсутствия переходов за пределы метода и т.д. При этом структура метода имеет свою специфику, соответствие которой должно проверяться верификатором: последовательность блоков кода, перемежающихся с обработчиками исключений, поддерживая даже вложенные конфигурации обработчиков исключений, когда внутренний обработчик находится внутри внешнего. Однако эта гибкость уравнивается строгим набором правил, регулирующих переходы потока управления.

- Переходы из обработчика в код: безусловные переходы из обработчика исключений обратно в обычный сегмент кода запрещены. Это предотвращает произвольное возобновление нормального выполнения обработчиками исключений, вместо этого направляя их либо на повторное возбуждение исключения, явное завершение метода, либо на тщательно определенный путь восстановления, который не входит в основной поток кода в произвольной точке.



Рис. 1: Пример некорректного перехода.

9.2.2 Абстрактная интерпретация

Абстрактная интерпретация представляет собой формальный метод статического анализа программ, предназначенный для аппроксимации их поведения с целью доказательства корректности определённых свойств без выполнения программы на конкретных входных данных.

Метод был предложен Патриком и Радией Кюсо в 1970. Абстрактная интерпретация позволяет построить корректное приближение семантики программы, охватывающее все возможные её исполнения.

В процессе данного анализа происходит работа не с конкретными значениями, а с типами. При этом верификаторная система типов может отличаться от системы типов среды исполнения, ниже приведен пример системы типов верификатора целевой платформы. При этом отношение прототипирования для классов, определяемых пользователем, осуществляется посредством механизмов среды исполнения.

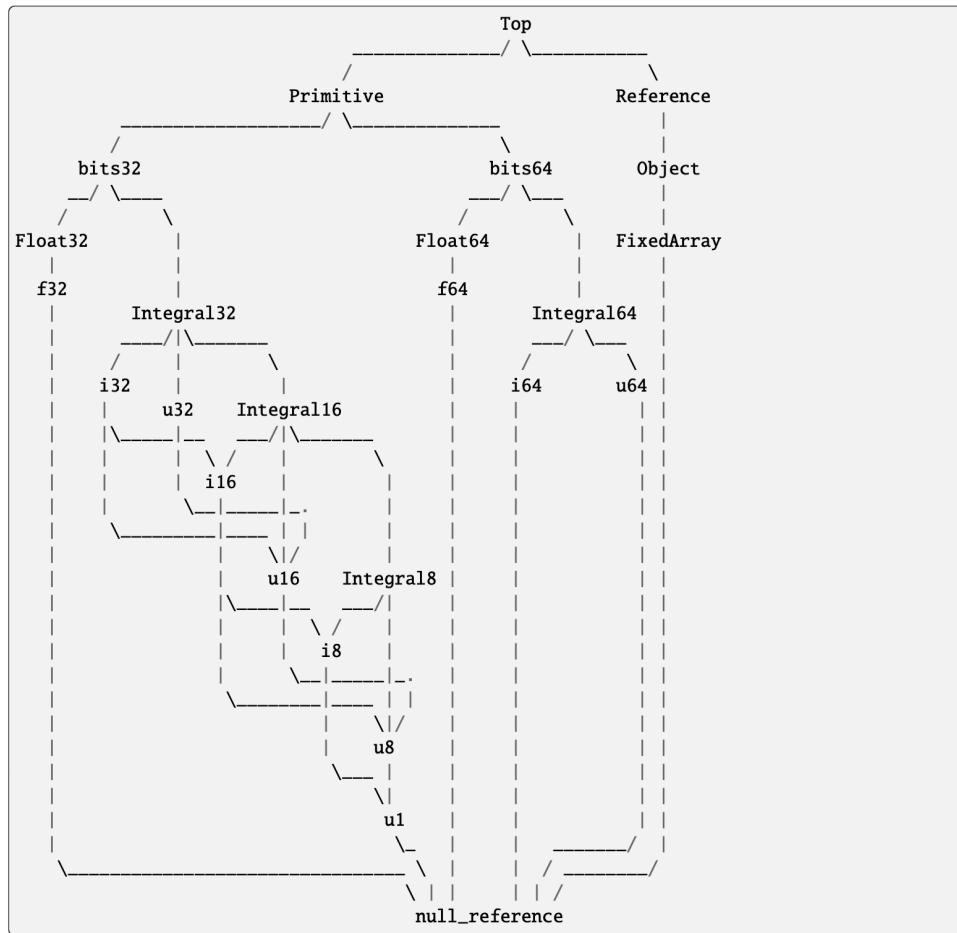


Рис. 2: Типовая система верификатора исследуемой платформы.

Алгоритм абстрактной интерпретации должен обладать следующими свойствами: *конечность* - анализ должен завершаться за конечное время. *достоверность* - если верификатор успешно отработал, то ошибка времени исполнения не может возникнуть, *неполнота* - то есть верификатор может отвергнуть валидный код. Разберем первое свойство более подробно.

Пусть программа P обладает конкретной семантикой, которая описывает точное поведение программы при всех возможных входных данных. Конкретная семантика, как правило, оперирует бесконечными множествами состояний, что делает прямой анализ вычислительно неразрешимым, т.к количество возможных состояний растет экспоненциально. Для решения данной проблемы вводится абстрактная семантика, которая: оперирует элементами абстрактного домена, является конечной, корректной аппроксимацией конкретной семантики. Между конкретной и абстрактной семантиками определяются функции:

- функция абстракции $\alpha : S_c \rightarrow S_a$;
- функция конкретизации $\gamma : S_a \rightarrow S_c$.

образующие галуасово соединение. По определению галуасово соединение

это:

$$\begin{aligned} & [\forall c \in C \wedge \forall a \in A] \mapsto \\ & \mapsto [\alpha(c) \sqsubseteq_A a \iff c \sqsubseteq_C \gamma(c)] \end{aligned}$$

,где A и C — некие абстрактный и конкретный домены. Выполнение данного условия является достаточным для гарантирования корректности верификации. Рассмотрим, как данная теория отражена на практике.

Примерами *абстрагирования* могут послужить операции *соединения* (*join*) и *расширения* (*widening*)

- Оператор *соединения* ($\sqcup$) представляет собой наименьшую верхнюю грань в абстрактном домене. Он заключается в объединении информации из различных путей выполнения программы для обеспечения наименее грубой аппроксимации, покрывающей несколько состояний. В алгоритмах абстрактной интерпретации *join* применяется в точках слияния графа потока управления. Программно это может выражаться в следующем: в одной ветке регистр v_1 имеет тип *String*, а по другой тип *int*. В итоге после слияния может произойти следующее:

$$String \sqcup Int = Object$$

если подразумевается, что *Object* — общий супертип (здесь и далее под супертипом подразумевается базовый класс). Или же возможно образование специального типа *объединение* (*union*)

$$String \sqcup Int = (String|Int)$$

- Для обеспечения завершения абстрактной интерпретации при анализе программ с циклами используется оператор *расширения* (∇). Расширение представляет собой специальную бинарную операцию над элементами абстрактного домена, ускоряющую достижение неподвижной точки путём ослабления точности аппроксимации. Применение расширения гарантирует завершение анализа за конечное число шагов, что является критически важным свойством для практических верификаторов байт-кода. Рассмотрим пример, когда используется данный подход:

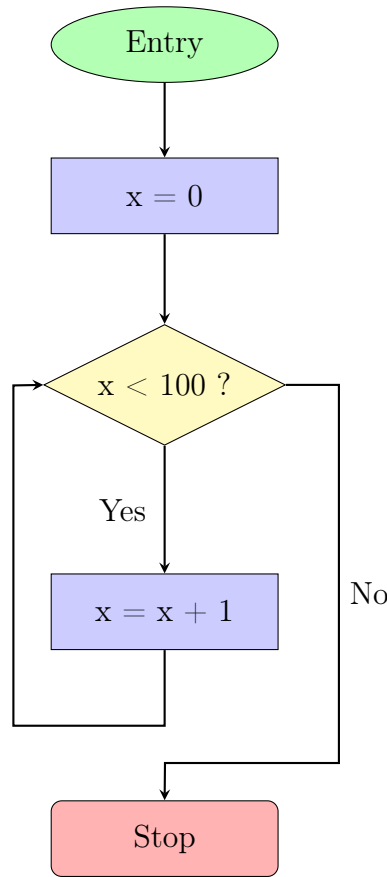


Рис. 3: Блок-схема цикла с условием для демонстрации операции *расширения*.

Используем интервальный домен:

$$x \in [l, r]$$

$$[l_1, r_1] \sqsubseteq [l_2, r_2] \Leftrightarrow l_2 \leq l_1 \wedge r_1 \leq r_2$$

На обратной дуге мы заменили возрастающую цепочку принимаемых x значений:

$$[0, 0] \sqsubseteq [0, 1] \sqsubseteq [0, 2] \sqsubseteq \dots$$

Данная цепочка может возрасть долго, поэтому домен состояний x будет определен как $[0, +\infty]$

Далее перейдем к рассмотрению байт-кода виртуальной машины и детально проанализируем несколько классов инструкций.

9.3 plugin opcodes

В исследуемой платформе часть семантики языка не может быть эффективно выражена только с помощью базового набора объектных инструкций наподобие обычных загрузок полей, вызовов виртуальных методов и сравнений ссылок. По этой причине в ISA выделено отдельное расширение с собственным префиксом инструкций. Такое расширение позволяет сделать байт-код ближе к исходной семантике языка и явно зафиксировать в коде те операции, которые зависят не только от низкоуровневого представления объектов, но и от языковых правил.

9.3.1 Мотивация появления плагиноспецифических инструкций

Одной из отличительных особенностей целевой платформы по сравнению с классическими java-подобными виртуальными машинами является поддержка объединения типов. Под объединением типов в данной работе понимается ссылочный тип, принимающий значения одного из нескольких классов. Для такого значения необходимо уметь безопасно обращаться к общим членам объединения.

```
1  class A {  
2      foo() {  
3          return 1  
4      }  
5  }  
6  class B {  
7      foo() {  
8          return 2  
9      }  
10 }  
11  
12 function bar(v : A|B) {  
13     v.foo()  
14 }
```

Листинг 1: Пример кода с объединением типов

```
1  lda.obj a0  
2  sta.obj v1  
3  plugin.call.name.short union.%%union_prop-A|B.foo:(union.%%union_prop-A|B), v1  
4  return.void
```

Листинг 2: Представление кода из Листинга 1 на уровне байткода

Компилятор генерирует синтетическое описание объединения и на этапе кодогенерации строит инструкцию обращения по имени к общему члену объединения. Важно, что язык не допускает создания объекта типа объединения через оператор `new`. Следовательно, во время исполнения в регистре может находиться исключительно экземпляр конкретного класса, например `A` или `B` (см. листинг 1), а инструкция задает не адрес конкретного слота, а имя требуемого поля или метода. Таким образом, `plugin`-инструкции позволяют перенести операцию разрешения члена на время исполнения.

9.3.2 Основные семейства `plugin`-инструкций

Содержательно `plugin`-расширение можно разбить на несколько семейств:

- инструкции чтения поля по имени: `plugin.ldobj.name`, `plugin.ldobj.name.64`, `plugin.ldobj.name.obj`;

- инструкции записи поля по имени: `plugin.stobj.name`, `plugin.stobj.name.64`, `plugin.stobj.name.obj`;
- инструкции вызова метода по имени: `plugin.call.name.short`, `plugin.call.name`, `plugin.call.name.range`;
- специальные инструкции для работы с языковым значением `nullvalue`: `plugin.ldnullvalue`, `plugin.movnullvalue`, `plugin.isnullvalue`;

Первое семейство отвечает за чтение и запись поля объекта по имени. По сравнению с обычными `ldobj/stobj` здесь важен не только идентификатор имени, но и то, что итоговое разрешение выполняется относительно фактического класса объекта-получателя. Существование вариантов `.64` и `.obj` отражает различие в представлении результата: отдельно обрабатываются 64-битные примитивы и ссылочные значения, тогда как базовый вариант используется для 32-битных значений.

С точки зрения реализации такая инструкция интересна тем, что рантайм сначала пытается найти реальное поле с требуемым именем в классе объекта и его базовых классах. Если подходящее поле отсутствует, то доступ может быть интерпретирован как вызов аксессуара. Иными словами, операция по имени абстрагируется от конкретной формы представления члена: логически это может быть либо поле, либо `getter/setter`-метод с согласованным именем. Такой подход позволяет сохранить естественную объектную модель языка на уровне байткода, не сводя все обращения исключительно к полям.

Второе семейство, `plugin.call.name.*`, предназначено для вызова метода по имени. В отличие от обычного виртуального вызова, где компилятор чаще всего уже знает точную сигнатуру члена, здесь существенен сам факт вызова по имени относительно текущего объекта. Формы `short`, `full` и `range` различаются способом передачи аргументов: короткая форма удобна для небольшого числа регистров, а `range`-вариант используется, когда аргументов больше и они расположены в непрерывном диапазоне регистров.

`plugin`-инструкции доступа по имени не являются исключительно механизмом для объединений типов. Они задают обобщенный способ работы с объектной семантикой `plugin`, когда конкретный член определяется именем и текущим фактическим классом объекта. Это делает их полезными и в иных языковых конструкциях, где требуется отложенное разрешение члена.

9.3.3 Специальные `plugin`-инструкции

Помимо доступа к полям и вызовам, `plugin`-расширение содержит инструкции, отражающие специальные языковые понятия. Семейство `nullvalue` включает три инструкции: `plugin.ldnullvalue`, `plugin.movnullvalue`,

`plugin.isnullvalue`. Они работают со специальным ссылочным значением `nullvalue`, которое используется для представления nullish-состояния в семантике языка. Наличие отдельного семейства инструкций показывает, что для платформы важно различать обычные объектные ссылки и специальное пустое значение, значимое на уровне языка.

Таким образом, `plugin`-расширение можно рассматривать как промежуточный слой между общей объектной моделью виртуальной машины и высокоуровневой семантикой языка. Оно позволяет сохранить в байт-коде сведения о том, что операция возникла именно из `plugin`-конструкции, а не была сведена к набору универсальных низкоуровневых действий.

9.4 any opcodes

Другой важной особенностью исследуемой платформы является поддержка взаимодействия со средой динамически типизированных языков. Для исследуемой платформы это особенно актуально в сценариях межъязыкового взаимодействия, когда статически типизированный код вызывает функции динамической стороны, получает из неё значения и затем обращается к их свойствам или методам.

9.4.1 Модель значения `any`

В типовую систему платформы для таких сценариев вводится специальный тип `any`. Существенно, что `any` не является просто синонимом ссылочного типа `Object`. В `any` может содержаться как значение, пришедшее из динамической среды, так и значение статической части программы. С точки зрения семантики это может быть как примитив, так и объектная ссылка.

9.4.2 Архитектурная роль значений `any`

Поддержка `any` не сводится к одному частному случаю доступа к свойству. Введение такого типа означает, что в архитектуре виртуальной машины явно фиксируется особый класс операций, связанных с передачей значений между статической и динамической семантикой. Иными словами, граница между статической и динамической частями программы становится видна непосредственно на уровне байткода.

9.4.3 Доступ к свойствам и элементам динамических значений

Непосредственное взаимодействие со значениями типа `any` осуществляется с помощью префиксного семейства `any.*`. Существуют два принципиально разных сценария. Если получатель является объектом межъязыкового взаимодействия, операция делегируется механизму `interop` с динамической средой. Если же получатель является обычным статическим

объектом, доступ обрабатывается внутри статической части рантайма через поиск поля или аксессора по имени.

```
1 // dynamic.js
2 class A {
3     prop = 1
4 }
5 function foo() {
6     return new A()
7 }
8
9 // static.ts
10 import {
11     A,
12     foo
13 } from "./dynamic.js"
14
15 let a = foo()["prop"]
```

Листинг 3: Пример взаимодействия со значением из динамической среды

```
1 lda.obj a0
2 sta.obj v1
3 any.ldbyname v1, "prop"
4 return.obj
```

Листинг 4: Представление кода из Листинга 3 на уровне байткода

Операции доступа можно разделить по способу адресации:

- чтение по имени свойства: `any.ldbyname`, `any.ldbyname.v`;
- запись по имени свойства: `any.stbyname`, `any.stbyname.v`;
- по числовому индексу: `any.ldbyidx`, `any.stbyidx`;
- по произвольному значению-ключу: `any.ldbyval`, `any.stbyval`.

Такое деление хорошо соответствует операциям динамических языков. Доступ по имени моделирует обращение к свойству объекта через строковый идентификатор. Доступ по индексу естественным образом подходит для массивоподобных структур. Наконец, доступ по значению-ключу отражает общую форму индексатора, где в качестве ключа может выступать произвольное значение.

При этом для объектов со статической стороны реализация имеет важную особенность. Если читаемое поле имеет примитивный тип, рантайм не возвращает «сырое» машинное значение наружу, а немедленно оборачивает его в соответствующий объект. Если же поле уже является ссылочным, его значение возвращается напрямую. Аналогично при записи значения через `any` в типизированное поле выполняется обратное преобразование: ссылочные значения записываются непосредственно, а для примитивных полей производится распаковка значения перед сохранением.

Именно эта логика и делает возможным единообразный интерфейс доступа к свойствам через `any` для полей разных типов.

Стоит заметить, что часть инструкций записывает результат в аккумулятор, а часть — непосредственно в регистр. Наличие двух вариантов делает ISA более гибким и позволяет избегать лишних промежуточных перемещений значения, если следующий этап вычисления ожидает аргумент именно в регистре.

9.4.4 Динамические вызовы

Помимо доступа к полям и индексаторам, семейство `any.*` покрывает основные формы вызова, характерные для динамического кода:

- вызов функции: `any.call.0`, `any.call.short`, `any.call.range`;
- вызов метода объекта по имени: `any.call.this.0`,
`any.call.this.short`, `any.call.this.range`;
- вызов конструктора: `any.call.new.0`, `any.call.new.short`,
`any.call.new.range`;
- проверка принадлежности типу: `any.isinstance`.

Виртуальная машина не скрывает динамическую семантику за одним универсальным вызовом, а различает сценарии явно. Вызов функции, вызов метода объекта и конструирование нового объекта выражаются разными байткодами. Для статических объектов вызов по имени в конечном счете сводится к поиску подходящего метода и, если его результат примитивен, к последующей упаковке этого результата в объектное представление. Данный подход облегчает как процесс исполнения, так и последующий анализ байткода. Разделение на формы `0`, `short` и `range` служит цели более эффективного исполнения каждого типа инструкций.

10 Описание практической части

Изучив базовый набор инструкций целевой виртуальной машины, перейдем непосредственно к реализации верификаторных обработчиков инструкций. Причем верификация должна быть реализована таким образом, чтобы выполнялись проверки на соответствие семантике, описанной в ISA.

Практическая часть работы была сосредоточена на расширении возможностей верификатора байт-кода для плагиноспецифичных инструкций и инструкций взаимодействия с динамической средой. В техническом плане работа состояла из четырех последовательных этапов. На первом этапе была подготовлена типовая инфраструктура для корректной работы с рантайм-объединениями. На втором этапе была реализована верификация инструкций семейства `plugin.call.name.*`. На третьем этапе было добавлено покрытие для множества `any.*`. Финальным этапом являлось тестирование реализованных обработчиков.

10.1 Подготовка типовой инфраструктуры для рантайм-объединений

Первым этапом был внесен подготовительный набор изменений, необходимый для более точного отображения рантайм `union` в типовой системе верификатора. До этого в коде верификатора существовали места, где `union`-тип фактически обрабатывался приближенно: представлял собой тип `Object`, часть проверок именованных доступов к членам объединений были ослаблены, поскольку информация о составе `union` не отображалась в верификаторную модель достаточно точно.

В рамках этого этапа была переработана связь между рантайм-представлением `union` и внутренним типом верификатора. В частности, `union`-класс, существующий на уровне рантайма, перестал рассматриваться как обычный `class`-тип. Для него была добавлена отдельная ветвь логики, которая начала распознавать не только специальные `union`-значения верификатора, но и рантайм-классы, помеченные как `union`. Это позволило извлекать множество составляющих типов непосредственно из рантайм-описания класса и использовать их в операциях подтипизации и анализа сигнатур.

```
1 ClassLinkerContext *CoreClassLinkerExtension::GetCommonContext(Span<Class *>  
    classes)  
2 {  
3     ASSERT(!classes.Empty());  
4     auto *commonCtx = classes[0]->GetLoadContext();  
5  
6     size_t foundClassesNum = 0;  
7     while (!commonCtx->IsBootContext()) {  
8         for (auto *klass : classes) {  
9             if (commonCtx->FindClass(klass->GetDescriptor()) != nullptr) {  
10                 foundClassesNum++;  
11             }  
        }  
    }
```

```
12     }
13     if (foundClassesNum == classes.Size()) {
14         break;
15     }
16     commonCtx = GetBootContext();
17 }
18 return commonCtx;
19 }
```

Листинг 5: Пример кода для поиска общего контекста

Дополнительно была доработана работа с контекстами загрузки классов. Для `union`-типа важно, чтобы верификатор мог выбрать общий контекст, в котором корректно видны все составляющие классы объединения. С этой целью в расширение `class linker` был добавлен алгоритм поиска общего контекста загрузки для множества классов. Если общий контекст обнаружить не удастся, логика консервативно поднимается к `boot context` - это корневой контекст виртуальной машины, в котором находятся системные и глобально доступные классы платформы.

Корректность этого этапа была подтверждена отдельным `verifier`-тестом на сигнатуры `union`-типов. Смысл теста заключался в проверке того, что сигнатура метода, принимающего объединение типов, сопоставляется верификатором с рантайм-описанием `union` без перехода к излишне грубому общему супертипу.

10.2 Реализация верификации `plugin.call.name.*`

После подготовки типовой инфраструктуры была реализована верификация инструкций `plugin.call.name.short`, `plugin.call.name` и `plugin.call.name.range`. Эти инструкции используются для вызова метода по имени, в том числе в сценариях, связанных с объединениями типов. До внесения изменений соответствующие обработчики в абстрактном интерпретаторе оставались заглушками, поэтому верификатор не умел содержательно анализировать такой вызов.

Основное изменение было внесено в абстрактный интерпретатор. Для всех трех форм инструкции была реализована единая схема анализа через общий вспомогательный интерфейс. В рамках этой схемы верификатор:

1. извлекает целевой метод из кэша, сформированного при разборе задания верификации;
2. проверяет успешность разрешения `method_id`;
3. выполняет проверку нарушений правил доступа к вызываемому методу;
4. трактует первый аргумент как `this` и проверяет, что он является подтипом ссылочного типа;

5. проверяет совместимость остальных фактических аргументов с формальной сигнатурой метода;
6. устанавливает тип аккумулятора в тип возвращаемого значения вызываемого метода.

Для таких форм, как **short**, работающей с двумя регистрами, и **full**, работающей с четырьмя регистрами, набор аргументов извлекается из фиксированного числа регистров.

```
1  template <BytecodeInstructionSafe::Format FORMAT>
2  bool HandlePluginCallNameRange()
3  {
4      LOG_INST();
5      DBGBRK();
6      uint16_t vs = inst_.GetVReg<FORMAT, 0x00>();
7      Method const *method = GetCachedMethod();
8      if (method != nullptr) {
9          LOG_VERIFIER_DEBUG_METHOD(method->GetFullName());
10     }
11
12     Sync();
13     constexpr size_t INIT_SIZE = 5;
14     SmallVector<int, INIT_SIZE> regs;
15     for (auto regIdx = vs; ExecCtx().CurrentRegContext().IsRegDefined(regIdx);
16         regIdx++) {
17         regs.push_back(regIdx);
18     }
19     return CheckPluginCall<FORMAT>(method, Span {regs});
20 }
```

Листинг 6: Пример кода для обработки `plugin.call.name.range`

Для **range**-формы был реализован отдельный сбор диапазона регистров из текущего верификаторного контекста, см. листинг 6, что согласует проверку с соглашением о кодировании аргументов в ISA.

С практической точки зрения важным было решение не дублировать всю рантайм-семантику инструкции в верификаторе. Проверка была построена как консервативная статическая аппроксимация: на этапе верификации проверяются только те свойства, которые могут быть надежно установлены без исполнения программы. В частности, верификатор гарантирует, что первый аргумент, интерпретируемый как **this**, имеет ссылочный тип, что остальные фактические аргументы совместимы с формальной сигнатурой вызываемого метода, и что тип результата, помещаемого в аккумулятор, согласован с объявленным типом возвращаемого значения. На этапе исполнения остается позднее разрешение целевого метода по фактическому классу объекта: после извлечения описания метода требуется подобрать совместимую реализацию именно для того класса, который находится в регистре в момент исполнения. Данный сценарий особенно важен в контексте объединения типов, где статически известен лишь абстрактный

тип, а конкретный класс получателя определяется только в рантайме. Также в интерпретатор были вынесены проверки на отсутствие подходящего метода в фактическом классе и на случай, когда найденная реализация остается абстрактной.

Для подтверждения корректности были добавлены как unit-тесты, так и негативные ассемблерные программы. Положительные тесты покрывают все три формы инструкции: короткую, полную и диапазонную.

Таким образом, на этом этапе верификатор получил поддержку специфичного множества инструкций, ранее не покрытого статическим анализом.

10.3 Реализация верификации `any.*`

Третьим этапом была реализована поддержка семейства `any.*`, отвечающего за операции над значениями, проходящими через интерфейс взаимодействия со статической и динамической частями платформы. В ходе работы были доработаны правила в ISA, расширен набор проверок в абстрактном интерпретаторе и добавлен набор тестов.

Первое изменение коснулось самой спецификации верификации в `isa.yaml`. Для инструкций `any.ldbyidx` и `any.stbyidx` были уточнены типы индексных операндов: вместо `i32` в правилах проверки были зафиксированы `f64`-значения, что соответствует реальной семантике инструкции. Кроме того, для `any.isinstance` был уточнен тип результата в аккумуляторе: результат проверки должен интерпретироваться как `i32`.

В абстрактном интерпретаторе были добавлены две служебные функции: `CheckRegTypeNotNullRef()` и `CheckRegTypes()`. Первая выделяет типовую для этого семейства проверку случая, когда инструкция гарантированно приведет к `NullPointerException`. Вторая позволяет компактно проверять несколько регистров сразу, что существенно упростило код обработчиков.

После этого были реализованы обработчики следующих инструкций:

- вызовы функций и методов: `any.call.0`, `any.call.short`, `any.call.range`, `any.call.this.0`, `any.call.this.short`, `any.call.this.range`;
- вызовы конструкторов: `any.call.new.0`, `any.call.new.short`, `any.call.new.range`;
- доступ к свойствам и индексаторам: `any.ldbyname`, `any.ldbyname.v`, `any.stbyname`, `any.stbyname.v`, `any.ldbyidx`, `any.stbyidx`, `any.ldbyval`, `any.stbyval`;
- проверка типа: `any.isinstance`.

Для большинства этих инструкций структура верификации оказалась однотипной. Сначала выполняется синхронизация состояния абстрактной интерпретации. Затем проверяются типы операндов: ссылочные аргументы должны иметь ссылочный тип, индексные аргументы — тип `f64`, а аккумулятор должен содержать значение ожидаемого типа. После этого выполняется отдельная проверка на случай, когда значение объекта-приемника заведомо является нулевым. Если такая ситуация обнаруживается, обработчик завершает анализ с фиксацией случая `AlwaysNpe`. Иначе верификатор обновляет тип результата: либо задает тип аккумулятора, либо выполняет переход к следующей инструкции.

Стоит отметить, что реализация была построена максимально единообразно. Семейство `any.*` содержит много инструкций с близкой семантикой, поэтому выделение общих проверок позволило избежать большого количества повторяющегося кода.

10.4 Тестирование результатов

Для инструкций `any.*` был добавлен отдельный набор unit-тестов. Этот набор покрывает как положительные, так и отрицательные сценарии. В него вошли тесты на вызовы функций и методов, конструкторов, чтение и запись по имени, чтение и запись по индексу, доступ по значению-ключу и инструкцию `any.isinstance`. Отдельные негативные тесты проверяют несовместимость типов, например использование неверного типа индексного операнда.

Также в тестовую инфраструктуру была внесена важная доработка: была реализована удобная инфраструктура для тестирования новых верификаторных обработчиков инструкций, которая упростила процесс проверки и обеспечила корректное выполнение verifier-тестов.

10.5 Итоги практической части

Итогом практической части стало расширение множества инструкций, для которых верификатор способен выполнять содержательную проверку типов и состояний регистров. Работа была построена поэтапно: от подготовки типовой модели объединенных типов, через добавление поддержки плагиноспецифичных вызовов, к реализации крупного блока проверок для `any.*`. Такой порядок оказался принципиально важным, поскольку последующие изменения опирались на корректное отображение рантайм- типов в модель верификатора.

С практической точки зрения проделанная работа улучшила две ключевые характеристики системы.

- **Безопасность:** был увеличен класс верифицируемых статически программ.

- **Совместимость:** повысилась точность верификации за счет более полного соответствия между рантайм-семантикой платформы и внутренней типовой системой верификатора.

11 Заключение

Обратимся к целям и задачам, сформулированным в начале работы, и проанализируем полученные результаты.

Основной целью работы было расширение типовой системы и увеличение множества проверяемых инструкций. Поставленная цель была достигнута за счет грамотной поэтапной реализации решения.

В ходе работы были получены следующие результаты.

1. Были изучены верификаторы байт-кода статически и динамически типизированных виртуальных машин. Разобраны основные этапы проверки метода: построение графа потока управления, создание верификаторного контекста, абстрактная интерпретация инструкций. Рассмотрены свойства статической проверки: консервативность анализа, конечность алгоритма, согласование абстрактной типовой модели с семантикой времени исполнения.
2. Проанализированы особенности plugin-расширения набора инструкций. Было уделено внимание инструкциям вызова метода по имени `plugin.call.name.*`, которые используются в сценариях, где конкретная реализация метода зависит от фактического класса объекта во время исполнения.
3. Проанализировано множество инструкций `any.*`. Было показано, что значение типа `any` может содержать как значение, пришедшее из динамической среды, так и значение из статической части программы. Поэтому проверка таких инструкций должна учитывать не только общий факт ссылочного представления, но и особенности операций доступа к свойствам, индексаторам, вызовам функций и проверкам типа на границе между статической и динамической семантикой.
4. Была расширена типовая система верификатора. В неё был добавлен новый тип - `union`, представляющий собой объединение нескольких ссылочных типов.
5. Реализованы верификаторные обработчики инструкций вызова метода по имени. При этом проверки, зависящие от состояния программы во время исполнения, такие как позднее разрешение метода по фактическому классу объекта, обработка отсутствия подходящей реализации и проверка абстрактного метода, были оставлены на стороне интерпретатора.
6. Реализована верификация инструкций `any.*`. В работу вошли инструкции вызова функций и методов, вызова конструкторов, чтения и записи свойств по имени, индексу и значению-ключу, а также инструкция

`any.isinstance`. Для этих инструкций также были уточнены правила в ISA.

7. Разработанные изменения были покрыты тестами: были добавлены положительные и отрицательные сценарии, покрывающие разные формы передачи аргументов. Также была доработана тестовая инфраструктура для запуска тестов верификатора в управляемом контексте исполнения.

Практическая значимость работы заключается в том, что верификатор стал проверять больший класс программ: с плагиноспецифическими инструкциями и операциями над значениями типа `any`. Это уменьшило число ситуаций, когда корректный байт-код не может быть содержательно проверен до исполнения, и повысило точность анализа за счет более полного соответствия между типовой моделью верификатора и моделью времени исполнения платформы.

Таким образом, поставленные задачи были выполнены. Результаты работы были добавлены в программный продукт.

Список литературы

- [1] ArkTs Documentation. Официальный сайт спецификации языка программирования ArkTs [Электронный ресурс]. — 2026. https://gitcode.com/igelhaus/arkcompiler_runtime_core/releases/.
- [2] Ecma Documentation. Официальный сайт спецификации языка программирования EcmaScript [Электронный ресурс]. — 2026. <https://tc39.es/ecma262>.
- [3] JVM Specification. Официальный сайт спецификации виртуальной машины JVM [Электронный ресурс]. — 2026. <https://docs.oracle.com/javase/specs/jvms/se8/html>.
- [4] .NET documentation. Официальный сайт исходного кода .NET виртуальной машины [Электронный ресурс]. — 2026. <https://github.com/dotnet/runtime/tree/main/docs>.
- [5] LLVM CFI verification tool [Электронный ресурс]. — 2026. <https://llvm.org/docs/CFIVerify.html>.
- [6] ART documentation. Официальный сайт исходного кода ART виртуальной машины [Электронный ресурс]. — 2026. <https://android.googlesource.com/platform/art/>.
- [7] *К.И., Владимиров / Владимиров К.И. // Оптимизирующие компиляторы. Структура и алгоритмы.* — АСТ, 2024.
- [8] *Blanchet, Bruno.* Introduction to Abstract Interpretation [Электронный ресурс]. — 2002. <https://bblanche.gitlabpages.inria.fr/absint.pdf>.
- [9] Abstract Interpretation [Электронный ресурс]. <https://pages.cs.wisc.edu/~horwitz/CS704-NOTES/10.ABSTRACT-INTERPRETATION.html>.
- [10] Oracle. "Java Platform, Micro Edition (Java ME) Documentation." Oracle, [Электронный ресурс]. — 2026. <https://www.oracle.com/java/technologies/java-me.html>.
- [11] Oracle. "Java Platform, Standard Edition (Java SE) Documentation." Oracle [Электронный ресурс]. — 2026. <https://docs.oracle.com/javase/>.
- [12] TypeScript. (n.d.). TypeScript: Handbook. TypeScript, n.d. — 2026. <https://www.typescriptlang.org/docs/handbook/intro.html>.

- [13] ISO International Standard ISO/IEC 14882:2020(E) – Programming Language C++ [Электронный ресурс]. — 2026. <https://isocpp.org/std/the-standard>.